

ĐẶC TẢ YÊU CẦU HỆ THỐNG P2P CHAT

1. YÊU CẦU CHỨC NĂNG (FUNCTIONAL REQUIREMENTS)

Hệ thống phải thực thi đầy đủ các nhiệm vụ cốt lõi của một **nút mạng ngang hàng (Peer-to-Peer Node)** chuyên dụng.

1.1. Mô hình Servent

- Mỗi ứng dụng khi khởi chạy phải đồng thời đảm nhiệm hai vai trò:
 - **Server:** Lắng nghe và chấp nhận các kết nối đến.
 - **Client:** Chủ động thiết lập kết nối tới các nút (peer) khác trong mạng.

1.2. Kết nối Peer

- Cho phép người dùng thiết lập kết nối tới các peer khác thông qua **định danh duy nhất**, bao gồm:
 - Địa chỉ IP
 - Số hiệu cổng (Port)

1.3. Gửi tin nhắn đa phương thức

- **Gửi tin 1-1:** Chuyển thông điệp tới một peer cụ thể được chọn trong danh sách kết nối.
- **Broadcast:** Gửi tin nhắn đồng thời tới toàn bộ các peer đang có trạng thái kết nối thành công.

1.4. Quản lý trạng thái thời gian thực (Real-time)

- Hệ thống phải cập nhật và hiển thị tức thời trạng thái của từng nút mạng, bao gồm:
 - Đang kết nối
 - Đã kết nối

- Lỗi
- Timeout
- Việc cập nhật trạng thái phải được thực hiện thông qua **cơ chế callback** và **xử lý đa luồng**, đảm bảo không làm treo giao diện.

1.5. Ngắt kết nối chủ động

- Người dùng có thể:
 - Chủ động ngắt kết nối với một peer bất kỳ.
 - Đóng toàn bộ ứng dụng, đảm bảo giải phóng tài nguyên mạng một cách an toàn.

2. YÊU CẦU PHI CHỨC NĂNG (NON-FUNCTIONAL REQUIREMENTS)

Các ràng buộc kỹ thuật nhằm đảm bảo **tính ổn định, an toàn và hiệu năng** của hệ thống.

2.1. Ràng buộc thời gian (Timeout)

- **Timeout kết nối:** Giới hạn tối đa **5 giây** để hoàn tất quá trình bắt tay TCP. Quá thời gian này, yêu cầu kết nối sẽ bị hủy.
- **Timeout gửi tin:** Giới hạn **5 giây** để đẩy dữ liệu vào luồng gửi, nhằm tránh tình trạng treo luồng khi mạng nghẽn.

2.2. Giới hạn dữ liệu

- Mỗi thông điệp ở lớp ứng dụng có độ dài tối đa **10.000 ký tự (10 KB)**.
- Mục đích: Ngăn chặn các cuộc tấn công từ chối dịch vụ (DoS) vào bộ nhớ hệ thống.

2.3. Tham số mặc định

- Sử dụng **cổng 12000** làm cổng lắng nghe tiêu chuẩn, trừ khi người dùng cấu hình giá trị khác.

2.4. Bảng mã ký tự

- Toàn bộ dữ liệu truyền tải phải được mã hóa theo chuẩn **UTF-8** để đảm bảo hiển thị chính xác tiếng Việt và các ký tự đặc biệt.

3. GIAO DIỆN & HÀNH VI NGƯỜI DÙNG (UI & BEHAVIOR)

3.1. Thành phần giao diện (GUI)

- Giao diện được xây dựng trên thư viện **Tkinter**, sử dụng **chế độ màu tối (Dark Mode)**.
- Các thành phần chính bao gồm:
 - Thanh công cụ kết nối (IP, Port, Nickname)
 - Nhật ký trò chuyện (ScrolledText)
 - Danh sách Peer trực quan
 - Vùng nhập tin nhắn kèm bộ đếm ký tự
 - Thanh trạng thái tổng quan ở phía dưới

3.2. Quy ước màu sắc trạng thái

- **Xanh lá:** Kết nối thành công
- **Vàng:** Đang trong quá trình bắt tay kết nối
- **Đỏ:** Ngắt kết nối hoặc xảy ra lỗi Socket
- **Cam:** Cảnh báo Timeout hoặc bộ đếm ký tự sắp chạm ngưỡng giới hạn

3.3. Luồng thao tác người dùng (User Flow)

1. Người dùng mở ứng dụng → Hệ thống tự động khởi tạo nút lắng nghe trên cổng được chỉ định.
2. Người dùng nhập IP/Port của peer đích → Nhấn **Kết nối** (hệ thống xử lý trên luồng riêng để tránh treo GUI).

3. Người dùng chọn một peer trong danh sách → Nhập tin nhắn → Nhấn **Gửi** (hoặc phím Enter).
4. Người dùng theo dõi phản hồi thông qua nhật ký trò chuyện hoặc thanh trạng thái hệ thống.

4. GIAO THỨC TRUYỀN THÔNG (COMMUNICATION PROTOCOL)

Quy định đóng gói thông điệp nhằm giải quyết vấn đề "**dính gói**" trong luồng byte TCP.

4.1. Cấu trúc thông điệp (Framing)

Mỗi gói tin được gửi đi bao gồm **hai phần**:

- **Header (4 byte):**
 - Chứa một số nguyên không dấu (định dạng **big-endian**).
 - Biểu diễn độ dài của phần nội dung tin nhắn.
- **Body:**
 - Chuỗi dữ liệu thô đã được mã hóa UTF-8.
 - Có độ dài đúng bằng giá trị được khai báo trong Header.

4.2. Xử lý tin nhắn rỗng

- Header có giá trị **0** được chấp nhận và được xem là một tin nhắn rỗng hợp lệ.

4.3. Xử lý vi phạm giới hạn

- Nếu Header khai báo độ dài lớn hơn **MAX_MESSAGE_LENGTH (10 KB)**:
 - Hệ thống phải **ngắt kết nối** với peer gửi tin.
 - Mục đích: Bảo vệ an toàn và ổn định cho hệ thống.

4.4. Đảm bảo tính toàn vẹn dữ liệu

- Đảm bảo đọc **đủ và chính xác** số byte cần thiết từ vùng đệm Socket trước khi tiến hành giải mã (decode) dữ liệu.